一、GNR-ABB-Siemens 自动化样品分析系统
技术手册与控制系统编程指导

本文件面向软件开发、PLC 编程和系统集成人员，提供可直接用于设计与编程的架构、流程、数据模型和实现要点。

---

1. 系统概述

本系统目标是实现铝锭（及其他金属）样品从进样、制样（铣削）、激发分析到归档的全自动处理，并完整实现标样架管理系统 **SCT（Setting-up / Control / Type Standardization）** 过程控制。

核心分工原则：

- **Host PC（上位机）**：承担主控计算机角色，负责样品档案、SCT 策略决策、流程编排、结果校正与数据管理。
- **西门子 PLC**：底层实时控制与安全互锁，协调机器人、激发台升降、铣床等执行单元。
- **ABB 1200 系列机器人**：样品抓取、搬运、旋转、放置。
- **GNR 仪器**：负责激发与原始数据输出（浓度、强度等），校正逻辑主要由 Host PC 完成。

---

2. 系统架构

text

[注册终端 / MES / 上位机界面]

↓

Host PC（策略引擎 + 样品管理 + 校正计算）

↓ (TCP/OPC UA/Modbus)

西门子 PLC（实时控制 + 安全）

↙ ↓ ↘

ABB 1200 激发台升降 铣床 PLC

机器人 (力控下压) (外部)

↓

GNR 光谱仪（激发 + 数据回传）

关键通信链路：

- Host PC ↔ PLC：任务下发、状态反馈、坐标/位置指令。
- PLC ↔ ABB 机器人：标准机器人通信（通常通过 Profinet 或机器人厂商接口）。
- Host PC ↔ GNR：激发触发、数据读取（浓度、强度）。
- Host PC ↔ 铣床 PLC：制样任务协调。

---

3. 主要硬件组成与功能

组件	型号/类型	主要功能	控制归属
机器人	ABB 1200 系列	抓取、搬运、旋转、放置	PLC 调度，机器人控制器执行
底层控制	西门子 PLC	逻辑控制、安全互锁、设备协调	实时执行层
光谱仪	GNR S6 系列	激发、数据采集	Host PC 触发与读取
激发台	已改造自动升降	力控下压样品	PLC 控制
铣床	下铣头方式	分析面铣削	外部 PLC，Host PC 协调
样品架	SCT 试样架 + 生产样品架	存放标样/控制样及生产样品	机器人操作，Host PC 管理位置
视觉系统	海康威视相机 + 高度传感器	选点、高度检测、位置确认	Host PC 处理图像
安全系统	安全围栏、急停、门禁	人员保护	PLC 硬互锁

---

4. 样品分类与档案结构

样品类型：

- 生产样品
- SCT 样品：
    - 设置样（Setting-up / 漂移校正样）
    - 控制样品（Control）
    - 标准样品 / 类型标准化样（Type Standardization）
    - 调节样品（可选）

样品档案核心字段（Host PC 数据库）：

字段	说明	备注
SampleID	唯一编号	主键
SampleType	生产 / SettingUp / Control / TypeStd	
AlloyType	关联合金类型	类型标准化匹配用
RackPosition	样品架位置编号	如 101、302
InitialThickness	初始厚度	mm
CurrentThickness	当前厚度	测高更新
MinThickness	最小允许厚度	报废阈值
UsageCount	已使用次数	
MaxUsageCount	最大允许次数	触发测高
Status	Normal / NeedMeasure / Warning / Disabled	
LastHeightCheck	上次测高时间	
AssociatedMethod	关联分析方法	
IsCRM	是否有证标准物质	

---

5. 核心工艺流程

5.1 生产样品自动分析流程

1. 样品注册（Host PC 或外部系统）。

2. 进样（手动门 / 传送带 / 缓冲架）。
3. 机器人抓取 → 视觉确认 / 高度检测（如需要）。
4. 送至铣床（下铣头，分析面朝下）进行铣削。
5. 铣削后机器人以“分析面以下固定距离”抓取。
6. 旋转使分析面朝下，放置到激发台预定位位置。
7. 激发台力控下压。
8. Host PC 判断是否需要插入 SCT 流程。
9. 触发 GNR 激发，读取强度/浓度。
10. Host PC 应用当前修正系数计算最终结果。
11. 归档（标签 + 归档盒）。
12. 更新样品轨迹日志。

举例：一个生产样品完整工艺流程

1. 样品注册
   └── Host PC 或外部系统录入样品信息（合金类型、分析方法等）
2. 样品进样
   └── 人工放入输入位 / 缓存台 / 传送带
   └── 传感器检测样品到位 → 通知 Host PC
3. 机器人抓取
   └── Host PC 下发抓取任务给 PLC
   └── 机器人按预设位置抓取（下铣头场景：固定相对高度）
   └── 可选：视觉确认抓取状态
4. 铣削制样（软件需预留此功能，本次因为没有配置铣床，是人工将铣削好的样品通过人工输入位输入，那么自动化系统将少一个铣削的样品流程，直接去视觉，其他的工艺流程不变。）
   └── 机器人将样品送至铣床
   └── 铣床（下铣头）对分析面进行铣削
   └── 完成后机器人以“分析面以下固定距离”重新抓取

5. 视觉选点（可选但推荐）

└── 机器人将样品送至视觉工位

└── 相机识别表面，规划激发点

└── 返回激发点坐标给 Host PC / PLC

6. 放置激发台

└── 机器人旋转使分析面朝下

└── 按计算高度放置到激发台预定位位置

└── 激发台力控下压，确保样品贴合且不移动

7. SCT 策略判断

└── Host PC 检查是否需要插入控制样 / 设置样标准化

└── 如需，则优先执行 SCT 流程，完成后继续

8. 激发分析

└── Host PC → GNR：CYCLE_START_REQUEST（ID_CYCLE=1 分析）

└── Host PC → GNR：SAMPLE_POSITIONED

└── GNR 完成激发

└── Host PC → GNR：RESULT_REQUEST + RESULT_DATA_REQUEST（获取强度/浓度）

9. 结果处理

└── Host PC 应用当前修正系数计算最终浓度

└── 判断是否合格（超差则报警或按策略处理）

└── 记录原始数据与校正后数据

10. 样品归档

└── 机器人取回样品

└── 激光打印标签

└── 放入归档盒

└── 更新样品档案与轨迹日志

11. 循环下一件

5.2 SCT 流程（由 Host PC 策略引擎驱动）

标准化（设置样）触发条件：

- 定时（班次开始 / 固定间隔）
- 切换到自动模式
- 控制样超差
- 用户 / 外部请求
- 类型标准化后漂移过大

执行步骤：

1. Host PC 选择状态正常的设置样。
2. 指令 PLC → 机器人从 SCT 架取出。
3. 测高（如使用次数达到阈值）。
4. 铣削（薄层）。
5. 放置激发台 → 激发。
6. Host PC 根据强度计算新修正系数并更新。
7. 样品放回原位，更新档案。

控制样流程：

- 按分析数量或时间插入。
- 超差 → 自动触发标准化 → 再用控制样验证。
- 连续失败则报警并暂停自动分析。

类型标准化：

- 合金类型切换或期限届满时触发。
- 匹配对应类型标样执行。

---

6. 上位机（Host PC）编程指导

6.1 推荐模块划分

模块	职责
SampleManager	样品档案、位置、使用次数、厚度管理
SCTStrategyEngine	触发判断、优先级调度、策略配置
WorkflowOrchestrator	流程状态机（生产样品 / SCT）
CorrectionEngine	强度读取、系数计算与应用
DeviceInterface	与 PLC、GNR、铣床的通信封装
VisionModule	相机图像处理、选点、高度检测
LogAndAlarm	全流程日志、报警管理

6.2 关键状态机（建议）

- **系统模式**：Manual / Auto / Maintenance / Emergency
- **任务状态**：Idle → Registered → Preparing → Milling → Placing → Analyzing → Archiving → Completed / Failed
- **SCT 子状态**：Selecting → Retrieving → MeasuringHeight → Milling → Analyzing → UpdatingCoeff → Returning

6.3 修正系数处理原则

- GNR 返回原始强度和浓度。
- 修正系数由 Host PC 计算并应用（优先方案）。
- 生产样品结果 = 应用最新系数后的值。
- 必须保存原始数据与校正后数据。

6.4 抓取与放置策略（结合前期讨论）

- 下铣头场景：机器人以分析面为基准，固定相对高度抓取。
- 标样变薄：抓取后先测高，动态调整抓取点。
- 激发台采用力控下压，避免样品移动。
- 视觉用于最终确认激发区域（可选但推荐）。

---

7. PLC 与机器人编程要点

- PLC 负责：安全互锁、激发台力控升降、设备就绪信号、机器人任务启停。
- 机器人任务由 PLC 触发，坐标或位置编号由 Host PC 下发。
- 建议使用标准工业协议（Profinet / OPC UA）保证实时性与可靠性。
- 所有动作需有超时与失败反馈。

---

8. 安全与模式管理

- 自动模式下安全门锁定。
- 急停后必须故障复位才能恢复。
- 漂移校正 / SCT 执行期间暂停生产样品分析。
- 样品位置必须与软件记录一致，否则拒绝切换自动模式。

---

9. 异常处理与日志要求

必须记录：

- 每一步动作的开始/结束时间
- SCT 触发原因
- 控制样超差详情
- 系数变更前后对比
- 样品厚度变化历史
- 所有报警与人工干预

---

10. 实施建议

1. 先实现样品档案 + SCT 策略引擎。
2. 再完成与 PLC 的任务交互和状态反馈。
3. 验证下铣头固定抓取 + 力控下压的稳定性。
4. 逐步加入视觉选点和标样自动测高。
5. 最后完善与 GNR 的数据接口和系数计算。

二、关于 sct 标样库的管理简介：

对应关系

中文名称	英文常用名称	说明
漂移校正样	Drift Correction Sample / Standardization Sample	强调功能：用于校正仪器漂移
设置样	Setting-up Sample（简称 SUS）	热电（Thermo ARL）SMS 系统中的标准叫法
标准化样品	Standardization Sample	通用叫法，与上两者同义
控制样品	**Control Sample**	**实时监控仪器分析状态是否正常**，判断是否发生了漂移或异常。

SCT 中有控制样（Control Sample）。

控制样的作用

控制样是 SCT 体系中的**监控样品**，主要作用是**实时监控仪器分析状态是否正常**，判断是否发生了漂移或异常。

项目	说明
核心功能	定期插入分析，检查仪器是否仍处于受控状态
判断依据	分析结果是否落在预设的控制限内（通常为 ±2σ 或 ±3σ）
与设置样的区别	设置样用于**校正**（计算并更新系数）；控制样用于**监控**（只判断是否合格，不直接改系数）
触发动作	控制样超差时，系统会自动或提示触发标准化（设置样校正）

典型使用方式

1. **按分析数量插入**：例如每分析 10 ~ 20 个生产样品后，自动插入一次控制样。
2. **按时间插入**：例如每班开始或每隔一定时间做一次控制样。
3. **超差处理流程**：
    - 控制样结果超差 → 触发设置样标准化
    - 标准化完成后，再次分析控制样进行验证
    - 验证通过则继续分析；仍超差则报警并暂停自动分析

在系统中的定位

样品类型	主要作用	是否更新系数
设置样（Setting-up）	漂移校正	是
控制样（Control）	状态监控	否（只判断）
类型标准化样	针对特定合金类型建立局部系数	是

总结

控制样相当于光谱分析中的"质控样品"，用来**发现问题**，而不是直接解决问题。
在你设计的 Host PC 系统中，控制样超差是触发自动标准化的重要条件之一。

三、铝合金类型和场景对应介绍：

1. 铝合金常见牌号体系（直读光谱常用）

系列	主要合金元素	常见牌号示例	典型用途
1xxx	纯铝（Al ≥ 99.00%）	1050、1060、1070、1100	电解铝原铝、导电用铝、包装箔
3xxx	Mn	3003、3004、3105	板带、罐料
5xxx	Mg	5052、5083、5182、5754	板带、船用、汽车板
6xxx	Mg+Si	6061、6063、6082	挤压型材（最常用）
7xxx	Zn+Mg	7075、7050	高强度结构件
8xxx	其他	8011 等	箔材

在**电解铝厂**中，日常分析主要集中在：

- **原铝（电解槽出铝）**：以 1xxx 系为主（高纯铝）
- **配料后的中间合金或成品铝锭**：常见 1xxx、3xxx、5xxx、6xxx 等

2. 电解铝行业过程样品与成品样品对应的标样情况

样品类型	典型合金范围	所需标样特点	大约数量
电解槽过程样品（原铝）	1xxx 高纯铝（Si、Fe、Cu、Mn、Mg、Zn 等杂质为主）	以杂质元素含量梯度为主的标样	**5–10 种**

样品类型	典型合金范围	所需标样特点	大约数量
成品铝锭 / 配料样品	1xxx + 部分 3xxx/5xxx/6xxx	覆盖主合金元素 + 杂质	**8–15 种**
合计（常规电解铝厂）	—	设置样 + 控制样 + 类型标样	**通常 10–20 种左右**

3. 实际标样配置建议（电解铝场景）

- **设置样（Setting-up / 漂移校正样）**：2–4 块
  通常选含量适中、稳定性好的样品，用于日常标准化。
- **控制样（Control Sample）**：3–6 块
  覆盖低、中、高不同杂质或合金含量水平，用于监控仪器状态。
- **类型标准化样（Type Standardization）**：按合金系列配置
  若只做原铝 + 少数成品，类型标样可控制在 5–10 种以内。
- **总标样数量**：大多数电解铝厂直读光谱系统配置 **12–25 块** 标样即可覆盖日常需求。
  若产品牌号较多（同时生产多种 6xxx、5xxx），可能增加到 30 块左右。

4. 与钢铁的对比

对比项	钢铁	电解铝（典型）
牌号数量	数百种	常用几十种，厂内常测的更少
标样总数	几十到上百块	**通常 10–25 块**
主要关注元素	C、Si、Mn、P、S、合金元素等	Si、Fe、Cu、Mn、Mg、Zn、Ti、V、Ga 等杂质 + 主合金元素
标准化频率	较高	相对稳定，但原铝杂质变化仍需定期校正

总结

在电解铝行业，针对电解槽过程样品和成品铝锭，**标样总数一般在 10–20 种左右**即可满足日常直读光谱分析需求。具体数量取决于：

- 实际生产的合金牌号数量
- 是否需要覆盖多个含量梯度
- 是否区分设置样、控制样和类型标样

四、SCT 样品库控制与选择逻辑
（完整设计原理 · 面向软件开发）

本文档说明 SCT 样品（设置样、控制样、类型标准化样）的档案管理、选择逻辑、使用次数与厚度控制、校正系数更新及数据记录的完整实现原理，便于软件人员进行编程。

---

1. 样品分类与档案结构

1.1 样品类型

类型	英文	主要作用	是否更新系数
设置样	Setting-up Sample	漂移校正	是
控制样	Control Sample	状态监控	否（仅判断）
类型标准化样	Type Standardization Sample	针对特定合金建立局部系数	是
调节样品	Adjustment Sample（可选）	微调	视情况

1.2 样品档案核心字段（数据库表建议）

text

```
SCT_Sample
├── SampleID              // 唯一编号
├── SampleName            // 名称
├── SampleType            // SettingUp / Control / TypeStd
├── AlloyType             // 关联合金类型（类型标样用）
├── RackPosition          // 样品架位置编号
├── CertifiedValues       // 标准值（JSON 或关联表）
├── InitialThickness      // 初始厚度 (mm)
├── CurrentThickness      // 当前厚度 (mm)
├── MinThickness          // 最小允许厚度
├── UsageCount            // 已使用次数
├── MaxUsageCount         // 最大允许次数（触发测高）
```

├── Status // Normal / NeedMeasure / Warning / Disabled

├── LastUseTime // 上次使用时间

├── LastHeightCheckTime // 上次测高时间

├── AssociatedMethod // 关联分析方法

├── IsActive // 是否启用

└── Remark

---

2. 样品入库与初始化

1. 人工将标样放入样品架指定位置。
2. 在 Host PC 中录入样品信息：
    - 基本信息、标准值、初始厚度、最大使用次数、最小厚度阈值。
3. 系统将 CurrentThickness = InitialThickness，UsageCount = 0，Status = Normal。
4. 样品位置与软件记录绑定，后续由机器人按位置抓取。

---

3. 使用次数与厚度控制逻辑

3.1 使用次数管理

- 每次成功完成激发（无论是标准化还是控制检查），UsageCount + 1。
- 当 UsageCount >= MaxUsageCount 时，将样品标记为 NeedMeasure。
- 下次被选中时，必须先执行测高流程。

3.2 测高与重新抓取逻辑

text

选中样品

↓

判断 UsageCount >= MaxUsageCount ?

├── 是 → 机器人抓取 → 放置测高台 → 读取高度

│ ↓

| 更新 CurrentThickness

| ↓

| 判断 CurrentThickness < MinThickness ?

| ├── 是 → Status = Disabled，报警更换

| └── 否 → 根据新厚度重新计算抓取高度，Status = Normal，UsageCount 可重置或继续累加

└── 否 → 按原抓取位置直接使用

3.3 抓取高度动态调整（下铣头场景）

- 机器人以"分析面以下固定距离"为基准抓取。
- 样品变薄后，通过测高结果重新计算抓取点，保证分析面始终能准确放到激发台。

---

4. SCT 样品选择逻辑

4.1 设置样选择

触发条件（由策略引擎判断）:

- 定时（班次开始 / 固定间隔）
- 控制样超差
- 切换到自动模式
- 用户手动请求
- 类型标准化后漂移过大

选择规则:

1. 筛选 SampleType = SettingUp 且 Status = Normal 的样品。
2. 优先选择与当前分析方法匹配的样品。
3. 若有多个，可按使用次数最少或轮询选择。
4. 若无可用样品，报警并暂停相关分析。

4.2 控制样选择

触发条件:

- 按分析数量（如每 N 个生产样品）
- 按时间间隔
- 标准化完成后验证

选择规则:

1. 筛选 SampleType = Control 且 Status = Normal。
2. 可按合金类型或含量梯度匹配。
3. 支持多块控制样轮流使用。

4.3 类型标准化样选择

- 当合金类型切换或类型标准化期限届满时触发。
- 按 AlloyType 精确匹配对应标样。

---

5. 校正系数更改逻辑

5.1 设置样标准化流程

1. 选择设置样 → 测高（如需要）→ 铣削 → 激发。
2. GNR 返回强度数据。
3. Host PC 根据强度与标准值计算新修正系数（斜率/截距）。
4. 更新当前方法的修正系数。
5. 记录系数变更日志（变更前、变更后、时间、操作原因）。

5.2 控制样超差处理

text

控制样分析完成

↓

结果是否在控制限内?

├── 是 → 记录结果，继续生产样品分析

└── 否 → 标记超差

↓

触发设置样标准化

↓

标准化完成后再次分析该控制样

↓

再次判断是否合格

├── 合格 → 恢复分析

└── 仍不合格 → 报警，暂停自动模式

5.3 系数应用原则

- 所有生产样品结果由 Host PC 应用最新修正系数计算。
- 必须同时保存**原始数据**和**校正后数据**。
- 系数变更需有完整审计日志。

---

6. 数据记录要求

每次 SCT 操作必须记录：

记录项	**说明**
样品 ID 与类型	哪块样品被使用
触发原因	定时 / 超差 / 手动等
使用次数变化	更新前后
厚度变化	测高前后
原始强度 / 浓度	GNR 返回值
校正系数	变更前后
控制样判定结果	合格 / 超差及偏差值
操作时间与操作者	审计用
机器人动作轨迹	可选，便于故障追溯

---

7. 软件模块划分建议

模块	职责
SCTSampleManager	档案管理、位置、次数、厚度、状态
SCTStrategyEngine	触发判断、样品选择、优先级调度
CorrectionEngine	系数计算、应用、版本管理
WorkflowOrchestrator	调用机器人、铣床、仪器的流程编排
AuditLogger	全流程日志记录

---

8. 关键状态机（SCT 样品）

text

Normal

├── 使用次数达到阈值 → NeedMeasure

├── 厚度低于阈值 → Disabled

└── 超差或异常 → Warning

NeedMeasure

└── 测高完成且厚度合格 → Normal

Disabled

└── 人工更换新样品并重新录入 → Normal

此逻辑完整覆盖了 SCT 样品库的控制、选择、使用次数与厚度管理、校正系数更新及数据记录，可直接作为软件详细设计的依据。

五、SCT 系统各模块详细设计
（系数计算、数据库表结构、状态机伪代码等）

---

1. 校正系数计算公式

直读光谱常用线性校正模型:

$$C_{校正} = a \times I + b$$

或强度比形式：

$$C_{校正} = a \times \frac{I_{元素}}{I_{内标}} + b$$

其中：

- I：测量强度（或强度比）
- a：斜率（Slope）
- b：截距（Intercept）
- $C_{校正}$：校正后浓度

1.1 设置样标准化系数计算（两点或单点）

单点标准化（常用）：

$$a_{new} = a_{old} \times \frac{C_{标准}}{C_{测量}}$$

或更常见的强度校正：

$$k = \frac{I_{标准期望}}{I_{当前测量}}$$

$$I_{校正} = I_{测量} \times k$$

然后用原工作曲线计算浓度。

两点标准化：

使用高、低两块设置样，重新拟合斜率 a和截距 b。

1.2 控制样判定

$$偏差 = \frac{C_{测量} - C_{标准}}{C_{标准}} \times 100\%$$

或绝对偏差。超限（通常 ±2σ 或设定百分比）即判定超差。

1.3 系数应用

生产样品:

$$C_{\text{最终}} = a_{\text{current}} \times I_{\text{测量}} + b_{\text{current}}$$

Host PC 保存原始强度与校正后浓度。

---

2. 数据库表结构

2.1 SCT 样品档案表 SCT_Sample

字段名	类型	说明
SampleID	VARCHAR(32)	主键
SampleName	VARCHAR(100)	名称
SampleType	VARCHAR(20)	SettingUp / Control / TypeStd
AlloyType	VARCHAR(50)	关联合金
RackPosition	VARCHAR(20)	架位编号
InitialThickness	DECIMAL(6,2)	初始厚度 mm
CurrentThickness	DECIMAL(6,2)	当前厚度
MinThickness	DECIMAL(6,2)	最小厚度
UsageCount	INT	已使用次数
MaxUsageCount	INT	触发测高阈值
Status	VARCHAR(20)	Normal / NeedMeasure / Warning / Disabled
LastUseTime	DATETIME	上次使用
LastHeightCheckTime	DATETIME	上次测高
IsActive	BIT	是否启用
CreatedTime	DATETIME	创建时间

2.2 标准值表 SCT_CertifiedValue

字段名	类型	说明
ID	BIGINT	主键
SampleID	VARCHAR(32)	外键
Element	VARCHAR(10)	元素
CertifiedValue	DECIMAL(10,4)	标准值
Unit	VARCHAR(10)	单位
Uncertainty	DECIMAL(10,4)	不确定度（可选）

2.3 校正系数表 CorrectionCoefficient

字段名	类型	说明
CoeffID	BIGINT	主键
MethodID	VARCHAR(50)	分析方法
Element	VARCHAR(10)	元素
Slope	DECIMAL(12,6)	斜率 a
Intercept	DECIMAL(12,6)	截距 b
EffectiveTime	DATETIME	生效时间
SourceSampleID	VARCHAR(32)	来源设置样
IsCurrent	BIT	是否当前有效
Operator	VARCHAR(50)	操作者

2.4 SCT 操作日志表 SCT_OperationLog

字段名	类型	说明
LogID	BIGINT	主键
SampleID	VARCHAR(32)	样品
OperationType	VARCHAR(30)	Standardization / ControlCheck / HeightMeasure
TriggerReason	VARCHAR(100)	触发原因

字段名	类型	说明
RawIntensity	TEXT	原始强度（JSON）
MeasuredConc	TEXT	测量浓度
CorrectedConc	TEXT	校正后浓度
IsPass	BIT	控制样是否合格
ThicknessBefore	DECIMAL(6,2)	测高前
ThicknessAfter	DECIMAL(6,2)	测高后
CoeffBefore	TEXT	变更前系数
CoeffAfter	TEXT	变更后系数
OperationTime	DATETIME	操作时间
ResultStatus	VARCHAR(20)	Success / Fail / Warning

2.5 控制限表 ControlLimit

字段名	类型	说明
LimitID	BIGINT	主键
SampleID	VARCHAR(32)	控制样
Element	VARCHAR(10)	元素
UpperLimit	DECIMAL(10,4)	上限
LowerLimit	DECIMAL(10,4)	下限
LimitType	VARCHAR(20)	Absolute / Percent / Sigma

---

3. 状态机伪代码

3.1 样品状态机

text

复制已复制

状态: Normal | NeedMeasure | Warning | Disabled

事件:

- UseSample　　// 被选中使用
- UsageThresholdHit　// 使用次数达阈值
- HeightMeasured　　// 完成测高
- ThicknessTooLow　　// 厚度低于下限
- ControlFail　　// 控制样超差
- ManualDisable　　// 人工禁用
- ReplaceSample　　// 更换新样品

转移:

Normal --UsageThresholdHit--> NeedMeasure

Normal --ControlFail--> Warning

NeedMeasure --HeightMeasured-->

　　if thickness >= MinThickness then Normal

　　else Disabled

Warning --StandardizationSuccess--> Normal

Disabled --ReplaceSample--> Normal

3.2 SCT 任务调度状态机

text

复制已复制

Idle

　↓ 触发条件满足

SelectingSample

　↓ 选中样品

CheckingSampleStatus

```
    ├── NeedMeasure → MeasuringHeight → UpdateThickness → Continue
    ├── Disabled → Alarm → Idle
    └── Normal → Continue
Milling
    ↓
Positioning
    ↓
Sparking
    ↓
ReadingResult
    ├── 设置样 → CalculateCoefficient → UpdateCoeff → Log → Idle
    └── 控制样 → EvaluateControl →
            ├── Pass → Log → Idle
            └── Fail → TriggerStandardization → Idle
```

---

4. 核心模块伪代码

4.1 样品选择（SCTStrategyEngine）

Python

复制已复制

```
def SelectSettingUpSample(methodId):
    candidates = Query(
        SampleType="SettingUp",
        Status="Normal",
        IsActive=True,
        AssociatedMethod=methodId
    )
    if not candidates:
```

```
        Alarm("无可用设置样")
        return None
    # 优先使用次数少的
    return Min(candidates, key=lambda s: s.UsageCount)


def SelectControlSample(methodId, alloyType=None):
    candidates = Query(
        SampleType="Control",
        Status="Normal",
        IsActive=True
    )
    # 可按合金类型过滤
    return RandomOrRoundRobin(candidates)
```

4.2 标准化执行

Python

复制已复制

```
def ExecuteStandardization(sample):
    if sample.UsageCount >= sample.MaxUsageCount:
        thickness = MeasureHeight(sample)
        UpdateThickness(sample, thickness)
        if thickness < sample.MinThickness:
            sample.Status = "Disabled"
            return Fail("样品过薄")

    Mill(sample)
    PlaceToSparkStand(sample)
    result = SparkAndGetData(sample)          # 调用 GNR
```

```
    oldCoeff = GetCurrentCoefficient(method)
    newCoeff = CalculateCoefficient(result, sample.CertifiedValues)
    SaveCoefficient(newCoeff, source=sample.SampleID)
    SetCurrentCoefficient(newCoeff)

    sample.UsageCount += 1
    LogOperation(sample, "Standardization", oldCoeff, newCoeff)
    return Success
```

4.3 控制样判定

Python

复制已复制

```
def EvaluateControlSample(sample, measuredData):
    limits = GetControlLimits(sample.SampleID)
    allPass = True
    for element in measuredData:
        conc = measuredData[element]
        limit = limits[element]
        if not (limit.Lower <= conc <= limit.Upper):
            allPass = False
            LogDeviation(element, conc, limit)

    sample.UsageCount += 1
    LogOperation(sample, "ControlCheck", isPass=allPass)

    if not allPass:
        TriggerStandardization()
```

return allPass

---

5. 关键流程时序

控制样超差自动校正：

1. 控制样分析完成 → EvaluateControlSample 返回 Fail
2. 策略引擎触发 ExecuteStandardization
3. 选择设置样 → 激发 → 更新系数
4. 再次分析原控制样进行验证
5. 验证通过 → 恢复生产样品分析；失败 → 报警并暂停

以上内容可直接用于详细设计与编码。

六、SCT 系统进阶设计说明

多元素系数矩阵、并发控制、与 PLC 交互报文

---

1. 多元素系数矩阵

直读光谱通常同时分析多个元素，每个元素独立计算校正系数，也可引入干扰校正（IEC）。

1.1 基本多元素线性模型

对每个元素 i：

$$C_i = a_i \cdot I_i + b_i$$

或强度比形式：

$$C_i = a_i \cdot \frac{I_i}{I_{\text{ref}}} + b_i$$

1.2 带干扰校正的模型（推荐）

$$C_i = a_i \cdot I_i + b_i + \sum_{j \neq i} k_{ij} \cdot C_j$$

其中 k_{ij}为元素 j对元素 i的干扰系数。

1.3 系数存储结构（建议）

text

复制已复制

CorrectionCoefficient

├─── MethodID

├─── Element

├─── Slope (a)

├─── Intercept (b)

├─── ReferenceElement　　// 内标元素（可空）

├─── IsCurrent

└─── Version

InterferenceCoefficient

├─── MethodID

├─── TargetElement　　// 被干扰元素

├─── InterferingElement　　// 干扰元素

├─── Coefficient (k)

└─── IsCurrent

1.4 多元素标准化计算流程

1. GNR 返回各元素强度 I_i。
2. 对设置样，已知标准浓度 C_i^{std}。
3. 对每个元素独立计算新斜率（单点法示例）：

$$a_i^{\text{new}} = a_i^{\text{old}} \times \frac{C_i^{\text{std}}}{C_i^{\text{meas}}}$$

4. 若启用干扰校正，可在标准化后重新计算或保持原干扰系数。

5. 全部元素系数更新后，标记为当前有效版本，旧版本归档。

1.5 生产样品应用

Python

复制已复制

```
def ApplyCorrection(intensities, methodId):
    coeffs = GetCurrentCoefficients(methodId)
    interferences = GetInterferenceCoefficients(methodId)

    # 先计算初步浓度
    conc = {}
    for elem in intensities:
        conc[elem] = coeffs[elem].Slope * intensities[elem] + coeffs[elem].Intercept

    # 应用干扰校正（迭代 1-2 次即可）
    for _ in range(2):
        for elem in conc:
            correction = 0
            for inter in interferences.Where(t => t.TargetElement == elem):
                correction += inter.Coefficient * conc[inter.InterferingElement]
            conc[elem] += correction

    return conc
```

---

2. 并发控制

Host PC 同时面对机器人调度、SCT 策略、用户操作、仪器通信，需避免冲突。

2.1 主要并发场景

场景	风险	控制方式
生产样品分析中触发 SCT	机器人资源冲突	任务队列 + 优先级
多个 SCT 同时触发	重复标准化	互斥锁
用户手动操作与自动任务	状态不一致	模式锁
系数更新与结果计算	读到中间状态	版本号 / 读写锁

2.2 推荐并发模型

任务队列（优先级）：

text

复制已复制

优先级（高 → 低）:

1. 紧急停止 / 故障恢复
2. 设置样标准化（控制样超差触发）
3. 控制样检查
4. 类型标准化
5. 生产样品分析
6. 样品归档 / 非关键任务

关键互斥资源：

Python

复制已复制

```
class ResourceLock:
    RobotLock          # 机器人占用
    SparkStandLock     # 激发台占用
    CoefficientLock    # 系数读写锁（读写分离）
    SampleArchiveLock  # 样品档案更新
```

2.3 系数更新并发控制

Python

复制已复制

```
# 写系数时加写锁
with CoefficientLock.Write():
    oldVersion = GetCurrentVersion()
    SaveNewCoefficients(...)
    SetCurrentVersion(oldVersion + 1)

# 读系数时加读锁（可并发）
with CoefficientLock.Read():
    coeffs = GetCurrentCoefficients()
    version = GetCurrentVersion()
```

2.4 任务调度伪代码

Python

复制已复制

```
class TaskScheduler:
    def SubmitTask(task):
        queue.Enqueue(task, priority=task.Priority)

    def Run():
        while True:
            task = queue.Dequeue()
            if not AcquireResources(task.RequiredResources):
                queue.Requeue(task)     # 资源不足则重新排队
                continue
            try:
                Execute(task)
            finally:
```

ReleaseResources(task.RequiredResources)

---

3. 与 PLC 的具体交互报文

Host PC 与西门子 PLC 之间建议采用 **OPC UA** 或 **Modbus TCP**。以下以结构化报文形式描述（可映射到具体协议）。

3.1 任务下发（Host PC → PLC）

字段	类型	说明
TaskID	UINT32	任务唯一编号
TaskType	UINT16	1:生产样品 2:设置样 3:控制样 4:测高 5:归档
SampleID	STRING	样品编号
RackPosition	STRING	样品架位置
GripHeight	FLOAT	抓取高度（mm，相对分析面）
NeedMill	BOOL	是否需要铣削
MillProgram	UINT16	铣削程序号
SparkCount	UINT16	需要激发次数
Priority	UINT16	优先级

示例（JSON 风格描述）：

JSON

复制已复制

```
{
  "TaskID": 10086,
  "TaskType": 2,
  "SampleID": "SUS-001",
  "RackPosition": "A03",
  "GripHeight": 12.5,
  "NeedMill": true,
```

"SparkCount": 3

}

3.2 状态反馈（PLC → Host PC）

字段	类型	说明
TaskID	UINT32	对应任务
Status	UINT16	0:空闲 1:执行中 2:完成 3:失败 4:暂停
CurrentStep	UINT16	1:抓取 2:铣削 3:放置 4:激发中 5:完成
ErrorCode	UINT16	错误代码
ActualGripHeight	FLOAT	实际抓取高度
RobotReady	BOOL	机器人就绪
SparkStandReady	BOOL	激发台就绪
SafetyOK	BOOL	安全状态正常

3.3 关键交互时序

设置样标准化：

1. Host PC → PLC：下发 TaskType=2 的任务
2. PLC → Host PC：Status=1，CurrentStep=1（抓取）
3. PLC 完成铣削、放置
4. PLC → Host PC：CurrentStep=4（激发中），SparkStandReady=true
5. Host PC → GNR：发起激发
6. GNR 完成后，Host PC 计算系数
7. Host PC → PLC：通知任务完成，可取回样品
8. PLC → Host PC：Status=2（完成）

测高任务：

1. Host PC → PLC：TaskType=4，指定样品位置
2. PLC 控制机器人将样品放到测高台
3. PLC → Host PC：返回 ActualHeight

4. Host PC 更新 CurrentThickness，重新计算 GripHeight
5. 通知 PLC 任务完成

3.4 错误与安全报文

ErrorCode	含义	处理建议
1001	机器人抓取失败	重试或报警
1002	样品未检测到	检查位置
2001	激发台下压异常	检查力控
3001	安全门打开	暂停所有任务
9001	急停	全部停止，等待复位

3.5 心跳与模式同步

- Host PC 与 PLC 每 1–2 秒交换心跳。
- 模式切换（Auto / Manual / Maintenance）必须双方确认。
- 任何一方检测到通信中断超过阈值，应进入安全状态。

---

4. 综合建议

模块	建议
系数	按元素独立存储，支持干扰校正，版本化管理
并发	优先级队列 + 资源锁，系数读写分离
PLC 交互	任务化报文，明确状态机与错误码
可靠性	所有关键操作记录日志，支持任务重试与人工干预

七、OPC UA 节点设计、干扰校正算法、完整任务状态机代码框架

---

1. OPC UA 节点设计（Host PC ↔ 西门子 PLC）

建议采用 OPC UA 作为 Host PC 与 PLC 的主要通信方式，节点按功能分组。

1.1 节点结构总览

text

复制已复制

```
Objects
└──── SCTSystem
      ├──── System
      │     ├──── Mode                    // Auto / Manual / Maintenance
      │     ├──── Heartbeat               //  双方心跳
      │     ├──── SafetyOK
      │     └──── EmergencyStop
      ├──── Task
      │     ├──── Command                 // Host PC → PLC  任务下发
      │     ├──── Status                  // PLC → Host PC  状态反馈
      │     └──── CurrentTaskID
      ├──── Robot
      │     ├──── Ready
      │     ├──── Busy
      │     ├──── ErrorCode
      │     └──── CurrentPosition
      ├──── SparkStand
      │     ├──── Ready
      │     ├──── Pressing
      │     ├──── ForceFeedback
      │     └──── ErrorCode
      └──── Devices
            ├──── MillReady
            └──── HeightSensorValue
```

1.2 关键节点定义

节点路径	数据类型	访问	说明
SCTSystem.System.Mode	Int16	R/W	0:Manual 1:Auto 2:Maintenance
SCTSystem.System.Heartbeat	UInt32	R/W	递增计数，超时判断
SCTSystem.Task.Command	Structure	W	任务下发（见下）
SCTSystem.Task.Status	Structure	R	任务状态反馈
SCTSystem.Robot.Ready	Boolean	R	机器人就绪
SCTSystem.SparkStand.Ready	Boolean	R	激发台就绪
SCTSystem.Devices.HeightSensorValue	Float	R	测高值

1.3 任务命令结构（TaskCommand）

text

复制已复制

```
TaskCommand
 ├─── TaskID              : UInt32
 ├─── TaskType            : Int16       // 1 生产  2 设置样  3 控制样  4 测高  5 归档
 ├─── SampleID            : String
 ├─── RackPosition        : String
 ├─── GripHeight          : Float
 ├─── NeedMill            : Boolean
 ├─── SparkCount          : UInt16
 ├─── Priority            : Int16
 └─── Reserved            : String
```

1.4 任务状态结构（TaskStatus）

text

复制已复制

```
TaskStatus
 ├─── TaskID                  : UInt32
```

├─── Status : Int16 // 0 空闲 1 执行中 2 完成 3 失败 4 暂停

├─── CurrentStep : Int16 // 1 抓取 2 铣削 3 放置 4 激发中 5 完成

├─── ErrorCode : Int16

├─── ActualGripHeight : Float

├─── Message : String

└─── Timestamp : DateTime

1.5 订阅与方法建议

- **订阅**：Task.Status、Robot.Ready、SparkStand.Ready、System.SafetyOK
- **方法（可选）**：
    - StartTask(TaskCommand)
    - AbortTask(TaskID)
    - ResetError()

---

2. 干扰校正具体算法

2.1 数学模型

对元素 i：

$$C_i = a_i \cdot I_i + b_i + \sum_{j \neq i} k_{ij} \cdot C_j$$

其中 k_{ij}为干扰系数。

2.2 迭代求解算法

Python

复制已复制

```
def ApplyInterferenceCorrection(rawConc, interferenceMatrix, maxIter=3, tolerance=0.0001):
    """
    rawConc: dict {element: preliminary concentration}
    interferenceMatrix: dict {(target, interferer): k}
```

```
    """
    conc = rawConc.copy()

    for iteration in range(maxIter):
        newConc = {}
        maxDelta = 0.0

        for elem in conc:
            correction = 0.0
            for (target, interferer), k in interferenceMatrix.items():
                if target == elem and interferer in conc:
                    correction += k * conc[interferer]
            newConc[elem] = rawConc[elem] + correction
            maxDelta = max(maxDelta, abs(newConc[elem] - conc[elem]))

        conc = newConc
        if maxDelta < tolerance:
            break

    return conc
```

2.3 干扰系数获取方式

方式	说明	适用场景
厂家提供	仪器或方法自带	首选
多标样回归	用多个标样拟合 k_{ij}	无现成系数时
经验值	根据文献或历史数据	临时使用

2.4 标准化时的处理建议

- **推荐**：标准化只更新斜率 a和截距 b，干扰系数 k_{ij}保持不变。
- 若漂移严重，可定期用类型标样重新验证干扰系数。

---

3. 完整任务状态机代码框架

Python

复制已复制

```
from enum import Enum, auto
from dataclasses import dataclass
from typing import Optional
import time


class TaskType(Enum):
    Production = 1
    SettingUp = 2
    Control = 3
    HeightMeasure = 4
    Archive = 5


class TaskStatus(Enum):
    Idle = 0
    Running = 1
    Completed = 2
    Failed = 3
    Paused = 4


class TaskStep(Enum):
    None_ = 0
```

```
    Picking = 1
    Milling = 2
    Placing = 3
    Sparking = 4
    Finished = 5


@dataclass
class Task:
    taskId: int
    taskType: TaskType
    sampleId: str
    rackPosition: str
    gripHeight: float
    needMill: bool
    sparkCount: int
    priority: int = 5
    status: TaskStatus = TaskStatus.Idle
    currentStep: TaskStep = TaskStep.None_
    errorCode: int = 0


class TaskStateMachine:
    def __init__(self, plcClient, gnrClient, sampleManager, correctionEngine):
        self.plc = plcClient
        self.gnr = gnrClient
        self.sampleMgr = sampleManager
        self.corrEngine = correctionEngine
        self.currentTask: Optional[Task] = None
```

```
    def Execute(self, task: Task):
        self.currentTask = task
        task.status = TaskStatus.Running

        try:
            # 1. 检查样品状态
            sample = self.sampleMgr.GetSample(task.sampleId)
            if sample.Status == "NeedMeasure" or task.taskType ==
TaskType.HeightMeasure:
                self._DoHeightMeasure(task, sample)

            if sample.Status == "Disabled":
                raise Exception("Sample disabled")

            # 2. 下发任务给 PLC
            self.plc.SendTask(task)

            # 3. 等待机器人完成放置
            self._WaitForStep(TaskStep.Placing)

            # 4. 根据任务类型处理
            if task.taskType in (TaskType.SettingUp, TaskType.Control,
TaskType.Production):
                self._DoSparkAndProcess(task, sample)

            task.status = TaskStatus.Completed
```

```
            task.currentStep = TaskStep.Finished

        except Exception as e:
            task.status = TaskStatus.Failed
            task.errorCode = self._MapError(e)
            self._HandleFailure(task)
        finally:
            self.plc.NotifyTaskEnd(task.taskId)
            self.currentTask = None

    def _DoHeightMeasure(self, task, sample):
        task.currentStep = TaskStep.Picking
        # PLC 执行测高
        height = self.plc.MeasureHeight(sample.RackPosition)
        self.sampleMgr.UpdateThickness(sample, height)
        if height < sample.MinThickness:
            sample.Status = "Disabled"
            raise Exception("Sample too thin")

    def _DoSparkAndProcess(self, task, sample):
        task.currentStep = TaskStep.Sparking

        # 通知 GNR 开始循环
        cycleType = self._MapCycleType(task.taskType)
        self.gnr.StartCycle(cycleType)

        # 通知样品已定位
```

```
        self.gnr.SamplePositioned(sample)

        # 等待结果
        result = self.gnr.GetResult()
        data = self.gnr.GetAnalysisData()    # 扩展接口

        if task.taskType == TaskType.SettingUp:
            self.corrEngine.UpdateCoefficient(data, sample)
        elif task.taskType == TaskType.Control:
            passed = self.corrEngine.EvaluateControl(data, sample)
            if not passed:
                self._TriggerStandardization()

        sample.UsageCount += 1
        self.sampleMgr.Save(sample)

    def _WaitForStep(self, targetStep, timeout=60):
        start = time.time()
        while time.time() - start < timeout:
            status = self.plc.GetTaskStatus(self.currentTask.taskId)
            if status.CurrentStep >= targetStep.value:
                return
            if status.Status == TaskStatus.Failed:
                raise Exception(f"PLC error: {status.ErrorCode}")
            time.sleep(0.5)
        raise TimeoutError("Wait step timeout")
```

```
    def _MapCycleType(self, taskType):
        return {TaskType.Production: 1, TaskType.SettingUp: 2, TaskType.Control: 3}.get(taskType, 1)

    def _TriggerStandardization(self):
        # 向调度器提交高优先级标准化任务
        pass

    def _HandleFailure(self, task):
        # 记录日志、报警、必要时安全复位
        pass

    def _MapError(self, exception):
        # 映射异常到错误码
        return 9999
```

---

4. 使用建议

模块	建议
OPC UA	优先使用结构化节点 + 订阅，减少轮询
干扰校正	先用初步浓度迭代 2–3 次，精度通常足够
状态机	将 PLC 交互与仪器交互分离，便于测试与故障恢复
扩展	可在此框架上增加重试、超时、人工确认等机制

八、**SCT 系统最终细化设计**

完整 OPC UA 信息模型 XML、干扰系数回归算法、与 GNR 协议结合的完整时序

---

1. 完整 OPC UA 信息模型 XML（简化版）

以下为可直接导入或作为参考的 NodeSet XML 结构（基于 OPC UA 标准）。

XML

复制已复制

```
<?xml version="1.0" encoding="utf-8"?>
<UANodeSet xmlns="http://opcfoundation.org/UA/2011/03/UANodeSet.xsd">
  <NamespaceUris>
    <Uri>http://yourcompany.com/SCTSystem</Uri>
  </NamespaceUris>

  <Aliases>
    <Alias Alias="Boolean">i=1</Alias>
    <Alias Alias="Int16">i=4</Alias>
    <Alias Alias="UInt16">i=5</Alias>
    <Alias Alias="UInt32">i=7</Alias>
    <Alias Alias="Float">i=10</Alias>
    <Alias Alias="String">i=12</Alias>
    <Alias Alias="DateTime">i=13</Alias>
  </Aliases>

  <!-- 对象类型 -->
  <UAObjectType NodeId="ns=1;i=1001" BrowseName="1:SCTSystemType">
    <DisplayName>SCTSystemType</DisplayName>
  </UAObjectType>

  <!-- 系统状态 -->
  <UAVariable NodeId="ns=1;i=2001" BrowseName="1:Mode"
ParentNodeId="ns=1;i=1001" DataType="Int16">
```

```
    <DisplayName>Mode</DisplayName>
    <Description>0:Manual 1:Auto 2:Maintenance</Description>
  </UAVariable>

  <UAVariable NodeId="ns=1;i=2002" BrowseName="1:Heartbeat"
ParentNodeId="ns=1;i=1001" DataType="UInt32">
    <DisplayName>Heartbeat</DisplayName>
  </UAVariable>

  <UAVariable NodeId="ns=1;i=2003" BrowseName="1:SafetyOK"
ParentNodeId="ns=1;i=1001" DataType="Boolean">
    <DisplayName>SafetyOK</DisplayName>
  </UAVariable>

  <!-- 任务命令结构 -->
  <UAVariable NodeId="ns=1;i=3001" BrowseName="1:TaskCommand"
ParentNodeId="ns=1;i=1001" DataType="Structure">
    <DisplayName>TaskCommand</DisplayName>
  </UAVariable>

  <!-- 任务状态结构 -->
  <UAVariable NodeId="ns=1;i=3002" BrowseName="1:TaskStatus"
ParentNodeId="ns=1;i=1001" DataType="Structure">
    <DisplayName>TaskStatus</DisplayName>
  </UAVariable>

  <!-- 机器人状态 -->
  <UAVariable NodeId="ns=1;i=4001" BrowseName="1:RobotReady"
```

ParentNodeId="ns=1;i=1001" *DataType*="Boolean"/>

<UAVariable *NodeId*="ns=1;i=4002" *BrowseName*="1:RobotErrorCode" *ParentNodeId*="ns=1;i=1001" *DataType*="Int16"/>

<!-- 激发台状态 -->

<UAVariable *NodeId*="ns=1;i=5001" *BrowseName*="1:SparkStandReady" *ParentNodeId*="ns=1;i=1001" *DataType*="Boolean"/>

<UAVariable *NodeId*="ns=1;i=5002" *BrowseName*="1:ForceFeedback" *ParentNodeId*="ns=1;i=1001" *DataType*="Float"/>

<!-- 测高传感器 -->

<UAVariable *NodeId*="ns=1;i=6001" *BrowseName*="1:HeightSensorValue" *ParentNodeId*="ns=1;i=1001" *DataType*="Float"/>

</UANodeSet>

实际工程建议：

- 使用 UaModeler 或类似工具生成完整 NodeSet。
- 将 TaskCommand 和 TaskStatus 定义为正式的 DataType（Structure）。
- 为关键变量配置 Historical Access（可选）。

---

2. 干扰系数回归算法

当没有厂家提供的干扰系数时，可通过多个标样回归计算。

2.1 数学模型

对目标元素 i，考虑干扰元素 j:

$$C_i^{\text{std}} = a \cdot I_i + b + k_{ij} \cdot C_j^{\text{std}} + \epsilon$$

或简化为多元线性回归:

$$C_i = \beta_0 + \beta_1 I_i + \beta_2 C_j + \epsilon$$

其中 β_2即为干扰系数 k_{ij}。

2.2 最小二乘回归实现

Python

复制已复制

```
import numpy as np
from sklearn.linear_model import LinearRegression


def CalculateInterferenceCoefficient(standards):
    """
    standards: list of dict
    [
        {"I_target": 1234.5, "C_interferer": 0.15, "C_target_std": 0.082},
        ...
    ]
    返回: k (干扰系数)
    """
    X = []
    y = []
    for s in standards:
        X.append([s["I_target"], s["C_interferer"]])
        y.append(s["C_target_std"])

    X = np.array(X)
    y = np.array(y)

    model = LinearRegression()
    model.fit(X, y)
```

```
    # model.coef_[0] -> 斜率 a
    # model.coef_[1] -> 干扰系数 k
    # model.intercept_ -> 截距 b
    return {
        "slope": model.coef_[0],
        "interference_k": model.coef_[1],
        "intercept": model.intercept_,
        "r2": model.score(X, y)
    }
```

2.3 使用建议

步骤	说明
1	准备 8–15 个含量梯度合理的标样
2	在稳定仪器状态下激发，记录强度与标准值
3	对每个目标元素分别回归
4	$R^2 > 0.95$ 时系数可用
5	定期用控制样验证干扰校正效果

---

3. 与 GNR 协议结合的完整时序

以**设置样标准化**为例，展示 Host PC、PLC、GNR 三者协同。

3.1 时序图（文字描述）

text

复制已复制

```
Host PC                                PLC                                GNR
   |                                    |                                  |
   |-- OPC UA: Send Task -------->|                                  |
```

```
|    (TaskType=SettingUp)          |                                |
|                                  |-- 机器人抓取/铣削/放置 --->|
|                                  |                                |
|<-- OPC UA: Step=Placing -----|                                |
|                                  |                                |
|-- TCP: CYCLE_START (ID=110) ------------------------->|
|    ID_CYCLE=2 (Standardization)                                |
|<-- TCP: Response OK ----------------------------------|
|                                  |                                |
|-- TCP: SAMPLE_POSITIONED (120) ---------------------->|
|                                  |                                |
|                                  |<-- 激发进行中 -------------|
|                                  |                                |
|-- TCP: RESULT_REQUEST (130) ------------------------->|
|<-- TCP: Standardization OK ----------------------------|
|                                  |                                |
|-- TCP: RESULT_DATA_REQUEST (扩展) ------------------->|
|<-- TCP: 浓度/强度数据 ---------------------------------|
|                                  |                                |
|-- 计算并更新系数                 |                                |
|                                  |                                |
|-- OPC UA: Task Complete ---->|                                |
|                                  |-- 取回样品放回样品架 ---->|
|<-- OPC UA: Status=Completed -|                                |
```

3.2 关键步骤伪代码

Python

复制已复制

```
def ExecuteSettingUpStandardization(sample):
    # 1. 下发任务给 PLC
    task = CreateTask(TaskType.SettingUp, sample)
    plc.SendTask(task)

    # 2. 等待样品放置完成
    WaitForPlcStep(task.TaskID, TaskStep.Placing)

    # 3. 与 GNR 通信
    gnr.Send(CYCLE_START_REQUEST, id_cycle=2)              # 标准化
    response = gnr.Receive()
    if response != OK:
        raise Exception("GNR refuse cycle")

    gnr.Send(SAMPLE_POSITIONED, spark_number=1)

    # 4. 等待并获取结果
    result_status = gnr.SendAndReceive(RESULT_REQUEST)
    if result_status not in (1, 3):    # OK 或 Warning
        raise Exception("Standardization failed")

    # 5. 获取详细数据（扩展接口）
    data = gnr.SendAndReceive(RESULT_DATA_REQUEST)

    # 6. 计算并更新系数
    UpdateCoefficients(data, sample.CertifiedValues)
```

```
# 7. 通知 PLC 完成
plc.NotifyComplete(task.TaskID)
WaitForPlcStatus(task.TaskID, TaskStatus.Completed)

# 8. 更新样品档案
sample.UsageCount += 1
SaveSample(sample)
LogStandardization(sample, data)
```

3.3 控制样超差完整闭环

text

复制已复制

1. 生产样品分析中插入控制样
2. Host PC → PLC：下发 Control 任务
3. PLC 完成放置 → Host PC → GNR：CYCLE_START (ID_CYCLE=3)
4. GNR 返回结果 → Host PC 判定超差
5. Host PC 立即提交高优先级 SettingUp 任务
6. 执行上述标准化流程
7. 标准化完成后，再次下发同一控制样验证
8. 验证通过 → 恢复生产样品队列

   验证失败 → 报警并暂停自动模式

---

4. 实施优先级建议

优先级	内容	说明
高	OPC UA 基本任务下发与状态反馈	先打通 Host PC 与 PLC
高	与 GNR 现有协议（100–140）对接	实现基本激发流程
中	RESULT_DATA_REQUEST 扩展	与 GNR 协商后实现

优先级	内容	说明
中	系数计算与应用	先做单元素，再扩展干扰校正
低	干扰系数回归工具	作为离线标定功能

以上内容已形成可落地的技术框架。